Docker and containers

Build small, secure images, run multi-container applications, and understand registries, runtimes and the OCI standard.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. Containers and images (15 min)
2. Writing good Dockerfiles (16 min)
3. Volumes and networking (15 min)
4. Multi-container apps with Compose (15 min)
5. Registries, tags and image supply-chain security (15 min)
6. Container runtimes, the OCI standard and Podman (14 min)
7. Container security and troubleshooting (16 min)
8. BuildKit, build caching and multi-architecture images (13 min)
9. Container logging, health checks and resource limits (13 min)
10. Slim images: smaller, faster and safer (12 min)
11. Running containers in production (13 min)
12. Debugging containers that will not start or keep dying (15 min)
Cheat sheet

Lesson 1. Containers and images

15 minutes. Understand what a container really is, how image layers work, and why pinning image versions prevents "it works on my machine".

A feature works on a developer's laptop, passes staging, and breaks in production. The code is identical. The difference is the environment: a different library version, a different base operating system, a different setting. Containers exist to remove that difference by shipping the application together with everything it needs.

What you will learn

- What a container is and how it differs from a virtual machine
- How images are built from layers
- The everyday `docker` commands
- Why floating tags such as `latest` cause surprises

A container is an isolated process

A container is **not** a small virtual machine. It is an ordinary Linux process that the kernel has isolated using two features:

- **Namespaces** give the process its own view of processes, network, mounts and hostname.
- **cgroups** limit how much CPU, memory and I/O it can use.

Because containers share the host's kernel, they start in well under a second and use far less memory than a VM, which runs a full operating system on virtual hardware. The trade-off is weaker isolation: a kernel flaw can affect every container on the host, so security settings matter (see the last lesson in this module).

	Virtual machine	Container
Includes	Guest operating system and kernel	Application and its libraries
Start time	Seconds to minutes	Milliseconds to seconds
Isolation	Strong, hardware level	Process level, shared kernel
Typical size	Gigabytes	Megabytes to a few hundred megabytes

Images and layers

An **image** is a read-only template. It is built from **layers**, where each instruction in a Dockerfile adds a layer on top of the last. A running **container** is an image plus a thin writable layer. When the container is removed, that writable layer disappears.

```
$ docker pull nginx:1.27
$ docker history nginx:1.27 | head -4
IMAGE          CREATED       CREATED BY                                      SIZE
a1b2c3d4e5f6   2 weeks ago   CMD ["nginx" "-g" "daemon off;"]                0B
<missing>      2 weeks ago   EXPOSE map[80/tcp:{}]                           0B
<missing>      2 weeks ago   COPY docker-entrypoint.sh / # buildkit          4.62kB
```

This is sample output; IDs and dates will differ. Layers are cached and shared. If ten images start from the same base, the base is stored once on disk, which is why pulls are fast after the first.

MARGA by EknathaLabs marga.eknathalabs.com

Everyday commands

```
$ docker run -d --name web -p 8080:80 nginx:1.27
$ docker ps
CONTAINER ID   IMAGE        COMMAND                  STATUS         PORTS                   NAMES
7f3a9c1b2d4e   nginx:1.27   "/docker-entrypoint.…"   Up 5 seconds   0.0.0.0:8080->80/tcp   web
$ curl -sI http://localhost:8080 | head -1
HTTP/1.1 200 OK
$ docker logs --tail 3 web
$ docker exec -it web sh
$ docker stop web && docker rm web
```

`-d` runs in the background, `--name` gives the container a readable name, and `-p 8080:80` publishes container port 80 on host port 8080. `docker exec` starts an extra process **inside** a running container, which is your debugging shell. `docker rm` deletes the container and its writable layer. Add `--rm` to `docker run` for throwaway containers that clean up after themselves.

Tags are labels, not versions

An image reference has a repository, a **tag** and optionally a **digest**: `nginx:1.27` or `nginx@sha256:...`. A tag is a movable label. The publisher can point `latest` or even `1.27` at a new image at any time. A digest identifies exact content and never changes.

Reference	Behaviour
`myapp:latest`	Whatever was pushed most recently, which can differ per host
`myapp:1.8.2`	A specific release, usually stable but still movable
`myapp@sha256:9b2...`	Exact bytes, immutable

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Containers and images fits together

Part	What it does	What goes wrong
A container is an isolated process	A container is not a small virtual machine.	Storing important data in the container's writable layer, which is lost on docker rm.
Images and layers	An image is a read-only template. It is built from layers, where each instruction in a Dockerfile adds a layer on top of the last.	Treating a container like a VM: logging in to change things by hand instead of rebuilding the image.
Everyday commands	-d runs in the background, --name gives the container a readable name, and -p 8080:80 publishes container port 80 on host port 8080.	Publishing ports you do not need to the whole network.
Tags are labels, not versions	An image reference has a repository, a tag and optionally a digest: nginx:1.27 or nginx@sha256:....	Using latest in production.

Flow: A container is an isolated process to Images and layers / Images and layers to Everyday commands / Everyday commands to Tags are labels, not versions

Incident: two servers run different code from the same tag

> **Incident:** *Production runs* `myapp:latest` *on three hosts. After a release, one host serves the new behaviour and two serve the old, and nobody changed the deploy command.*

`latest` was moved by a build on Monday. `docker run` and `docker compose up` reuse an image already stored on the host and only pull when it is missing. Hosts that had pulled `latest` before Monday kept the old image. Compare what each host actually runs:

```
$ docker image inspect myapp:latest --format '{{.Id}} {{index .RepoDigests 0}}'
sha256:3c1e... myapp@sha256:8f2a...
```

MARGA by EknathaLabs marga.eknathalabs.com

The digests differ between hosts, which proves it. The cure is to deploy by an immutable reference:

```
docker pull registry.example.com/myapp:1.8.2
docker run -d --name myapp registry.example.com/myapp:1.8.2
```

Have your pipeline tag each build with a version or commit identifier, deploy by that tag (or digest), and treat `latest` as a convenience for local experiments only. Rollback becomes trivial: run the previous tag.

Common mistakes

- Using `latest` in production
- Storing important data in the container's writable layer, which is lost on `docker rm`
- Treating a container like a VM: logging in to change things by hand instead of rebuilding the image
- Publishing ports you do not need to the whole network

Check yourself

What is a container?

Answer: An isolated process that shares the host kernel, limited by namespaces and cgroups

What happens to data written inside a container's writable layer when you run docker rm?

Answer: It is deleted with the container

Why is deploying myapp:latest in production risky?

Answer: The tag can move, so different hosts may run different content

What does docker run -p 8080:80 do?

Answer: Publishes container port 80 on host port 8080

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Writing good Dockerfiles

16 minutes. Use layer caching, multi-stage builds and a non-root user to build images that are fast to build, small and safer to run.

Every code change triggers a six-minute image build, because a dependency install runs again each time. The final image is 1.2 GB for an application that is a single 15 MB binary. Both are Dockerfile problems, and both have simple fixes once you understand how Docker builds images.

What you will learn

- The core instructions and what each does
- How layer caching works and how to order instructions for it
- Multi-stage builds for small final images
- How to run as a non-root user

The core instructions

Instruction	Purpose
`FROM`	The base image to start from
`WORKDIR`	Set the working directory for later steps
`COPY`	Copy files from the build context into the image
`RUN`	Execute a command at build time and save the result as a layer
`ENV`	Set an environment variable
`EXPOSE`	Document the listening port (it does not publish it)
`USER`	Switch to a non-root user
`CMD` / `ENTRYPOINT`	What runs when the container starts

`ENTRYPOINT` is the program, and `CMD` provides default arguments to it (or the whole command if there is no entrypoint). Use the **exec form** `["server", "--port", "8080"]` so the program is PID 1 and receives stop signals directly, which allows graceful shutdown.

How the cache works

Docker runs instructions in order and reuses a cached layer if the instruction and the files it depends on are unchanged. **The first changed layer invalidates every layer after it.** So put things that change rarely at the top and things that change often at the bottom.

```
# Slow: any source edit invalidates the dependency install
FROM python:3.12-slim
WORKDIR /app
COPY . .
RUN pip install --no-cache-dir -r requirements.txt
CMD ["python", "app.py"]

# Fast: dependencies are reinstalled only when requirements.txt changes
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

With the second version, editing `app.py` leaves the install layer cached:

```
$ docker build -t shop-api:dev .
 => CACHED [2/5] WORKDIR /app
 => CACHED [3/5] COPY requirements.txt .
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
 => [5/5] COPY . .
```

MARGA by EknathaLabs marga.eknathalabs.com

This is abbreviated sample output. A `.dockerignore` file keeps junk out of the build context and out of cache checks:

```
node_modules
.venv
*.log
.env
tmp/
```

> **Warning:** *Never `COPY` secrets such as `.env` files into an image. Every layer is stored and can be read by anyone who can pull the image, even if a later step deletes the file.*

Multi-stage builds

A build needs compilers and tools that the running application does not. A multi-stage build compiles in one stage and copies only the result into a small final stage.

```
FROM golang:1.23 AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -o /out/server ./cmd/server

FROM gcr.io/distroless/static-debian12:nonroot
COPY --from=build /out/server /server
USER nonroot
EXPOSE 8080
ENTRYPOINT ["/server"]
```

The first stage includes the whole Go toolchain, which is hundreds of megabytes. The final stage contains only the compiled binary on a minimal base with no shell or package manager, so it is both small (often under 20 MB) and exposes far less to an attacker. For interpreted languages, use a `-slim` base and install only runtime dependencies in the final stage.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Writing good Dockerfiles fits together

Part	What it does	What goes wrong
The core instructions	ENTRYPOINT is the program, and CMD provides default arguments to it (or the whole command if there is no entrypoint).	Using the shell form of CMD, which makes /bin/sh PID 1 and swallows stop signals.
How the cache works	Docker runs instructions in order and reuses a cached layer if the instruction and the files it depends on are unchanged.	Copying the whole source tree before installing dependencies.
Multi-stage builds	A build needs compilers and tools that the running application does not.	Leaving build tools and caches in the final image.

Flow: The core instructions to How the cache works / How the cache works to Multi-stage builds

Incident: a bloated, slow, root-running image

> **Incident:** *The platform team scans the shop image and flags a long list of vulnerabilities in tools nobody uses. The image is 1.2 GB, pulls take minutes on new nodes, and the container runs as root.*

Check what is inside:

```
$ docker image ls shop-api
REPOSITORY   TAG    IMAGE ID       SIZE
shop-api     old    5d0c1e2a9b77   1.21GB
shop-api     new    a4f7b8c3d1e0   18.4MB
$ docker run --rm shop-api:old id
uid=0(root) gid=0(root) groups=0(root)
$ docker run --rm shop-api:new id
```

MARGA by EknathaLabs marga.eknathalabs.com

```
uid=65532(nonroot) gid=65532(nonroot)
```

The old image used a full build image as its runtime and never set `USER`. The fix is the multi-stage Dockerfile above. The benefits are measurable: a 98% smaller image, faster pulls and rollouts, fewer packages to patch, and a process that cannot write to most of the filesystem if it is compromised. Add a **vulnerability scan** (for example with Trivy) to your pipeline so a new finding fails the build instead of reaching production.

Predict the layers

Common mistakes

- Copying the whole source tree before installing dependencies
- Using `latest` or an unpinned base image
- Running as root
- Leaving build tools and caches in the final image
- Using the shell form of `CMD`, which makes `/bin/sh` PID 1 and swallows stop signals

Check yourself

In a Dockerfile, why copy requirements.txt and install dependencies before copying the rest of the source?

Answer: So the dependency layer stays cached when only source files change

What is the main benefit of a multi-stage build?

Answer: The final image contains only what is needed at runtime, not the build tools

Why is the exec form of CMD (a JSON array) preferred?

Answer: The program runs as PID 1 and receives stop signals directly

You COPY .env into an image and delete it in a later RUN step. Is the secret safe?

Answer: No, the file remains readable in the earlier layer

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. Volumes and networking

15 minutes. Keep data beyond a container's life with volumes and bind mounts, and connect containers by name on user-defined networks.

A containerized app needs two things the container itself does not give you: a place to keep data that survives a restart, and a reliable way to find other containers. Get either wrong and you either lose data or spend an evening debugging why `db` does not resolve.

What you will learn

- The three ways to persist or share data
- How Docker networks and container DNS work
- Why the default network behaves differently from one you create
- How to publish ports safely

Persisting data

A container's writable layer is deleted with the container. Use one of these instead:

Type	Created with	Lives	Use for
Named volume	`-v pgdata:/var/lib/postgresql/data`	Managed by Docker	Databases and other app state
Bind mount	`-v /srv/config:/etc/app:ro`	A host path you choose	Config files, local development code
tmpfs	`--tmpfs /tmp`	Memory only	Scratch data that must not touch disk

```
$ docker volume create pgdata
$ docker run -d --name db -v pgdata:/var/lib/postgresql/data -e POSTGRES_PASSWORD=change-me postgr \
    es:16-alpine
$ docker rm -f db
$ docker run -d --name db2 -v pgdata:/var/lib/postgresql/data -e POSTGRES_PASSWORD=change-me postg \
    res:16-alpine
$ docker volume ls
DRIVER    VOLUME NAME
local     pgdata
```

The second database starts with the first one's data because the volume outlived the container. Back up a volume by running a throwaway container that mounts it and archives the contents, and **test restores**, as you learned in the scripting module.

> **Note:** *Bind mounts expose host paths directly. Add `:ro` for read-only wherever the container does not need to write, and remember that file ownership inside the container must match the host's numeric user ID.*

Networks and DNS

Containers on the same **network** can reach each other. Docker creates a default network called `bridge`, but it is best to create your own.

```
$ docker network create backend
$ docker run -d --name db --network backend -e POSTGRES_PASSWORD=change-me postgres:16-alpine
$ docker run --rm --network backend postgres:16-alpine pg_isready -h db
db:5432 - accepting connections
```

On a **user-defined network** Docker runs an embedded DNS server, so containers find each other **by name** (`db`). On the **default bridge** network there is no automatic name resolution between containers, only IP addresses, which change when containers restart.

	Default `bridge`	**User-defined bridge**
Name resolution	No	Yes, by container or service name

MARGA by EknathaLabs marga.eknathalabs.com

	Default `bridge`	**User-defined bridge**
Isolation	All containers share it	Only members can talk
Recommended	Quick experiments	Anything real

Publishing ports

Containers are reachable from other containers on their network without publishing anything. Publish a port only for traffic that must come from outside Docker.

```
docker run -d -p 8080:80 nginx:1.27                # all host interfaces
docker run -d -p 127.0.0.1:5432:5432 postgres:16   # localhost only
```

> **Warning:** `-p 5432:5432` *makes the database reachable from every network interface of the host. Docker also edits firewall rules itself, which can bypass a host firewall you thought was protecting you. Bind to* `127.0.0.1` *or do not publish the port at all.*

Incident: the app cannot find the database

> **Incident:** *The app container logs* `could not translate host name "db" to address: Name or service not known`*. Both containers are running.*

Check which network each container is on:

```
$ docker inspect -f '{{range $k, $v := .NetworkSettings.Networks}}{{$k}} {{end}}' app db
bridge
backend
```

`app` is on the default `bridge` network and `db` is on `backend`. They cannot see each other at all, and even if they shared the default bridge, name lookup would not work there. Put both on the same user-defined network:

```
$ docker network connect backend app
$ docker exec app getent hosts db
172.19.0.2      db
```

The command `getent hosts db` inside the app container proves the name now resolves. Make this permanent in the run command (`--network backend`) or, better, define the network in Compose (next lesson) so nobody has to connect things by hand.

Diagram: Where container data and traffic live

Part	What it does	What goes wrong
Container	A writable layer that disappears with the container.	Data written here is lost when the container is removed.
Volume	Managed storage that outlives the container.	Not mounted means no persistence.
Bind mount	A host folder mounted inside.	Wrong permissions or path on the host break the app.
Bridge network	Containers on it reach each other by name; the host publishes ports.	A container on another network cannot resolve the name.
Host and port	A published port maps to the container port.	Two containers cannot publish the same host port.

Flow: Container to Volume / Container to Bind mount / Container to Bridge network / Bridge network to Host and port

Common mistakes

- Keeping database files inside the container
- Publishing database ports to the whole host
- Relying on the default bridge network and container IP addresses
- Bind-mounting source code in production instead of baking it into the image

MARGA by EknathaLabs marga.eknathalabs.com

Check yourself

Which storage type is best for a database's data directory?

Answer: A named volume

Two containers are on the default bridge network. Can one reach the other by container name?

Answer: No, automatic name resolution is only available on user-defined networks

How do you publish a container port only on the host's loopback interface?

Answer: -p 127.0.0.1:5432:5432

What does the :ro suffix on a bind mount do?

Answer: Mounts it read-only inside the container

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. Multi-container apps with Compose

15 minutes. Describe an app, its database, networks, volumes and secrets in one file, and make services wait for real readiness.

Running an app with a database means a network, a volume, a password, a start order and about five `docker run` commands that someone must remember in the right order. Docker Compose captures all of that in one file that you can review, version and run with one command.

What you will learn

- The structure of a Compose file
- Health checks and start-up ordering
- Secrets and environment variables
- Everyday Compose commands

A complete example

```
services:
  app:
    build: .
    ports:
      - "8080:8080"
    environment:
      DB_HOST: db
      DB_USER: app
      DB_NAME: appdb
      DB_PASSWORD_FILE: /run/secrets/db_password
    secrets:
      - db_password
    depends_on:
      db:
        condition: service_healthy
    networks:
      - backend

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: app
      POSTGRES_DB: appdb
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    secrets:
      - db_password
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app -d appdb"]
      interval: 5s
      timeout: 3s
      retries: 10
    networks:
      - backend

secrets:
  db_password:
    file: ./secrets/db_password.txt

volumes:
  pgdata:

networks:
  backend:
```

Read it top to bottom. Each entry under `services` becomes a container. Compose creates the `backend` network and the `pgdata` volume, and services reach each other by their **service name** (`db`). Notice there is no `version:` line. Current Compose ignores that key and warns that it is obsolete.

MARGA by EknathaLabs
marga.eknathalabs.com

Secrets are mounted as files under `/run/secrets/` inside the containers that list them, so the password never appears in `docker inspect` output or in the image. The `postgres` image reads `POSTGRES_PASSWORD_FILE` natively. Your own application must read the file named by `DB_PASSWORD_FILE` for this to work, which is a small code change worth making.

Health checks and ordering

`depends_on` alone only waits for the container to *start*, not for the program inside to be *ready*. A database takes several seconds to accept connections. Adding a `healthcheck` and `condition: service_healthy` makes Compose wait until `pg_isready` succeeds before it starts `app`.

Everyday commands

```
$ docker compose up -d --build
$ docker compose ps
NAME            IMAGE           COMMAND                  SERVICE   STATUS                    PORTS
shop-app-1      shop-app        "/server"                app       Up 12 seconds             0.0.0 \
    .0:8080->8080/tcp
shop-db-1       postgres:16...  "docker-entrypoint.s…"   db        Up 17 seconds (healthy)   5432/ \
    tcp
$ docker compose logs -f --tail 20 app
$ docker compose exec db psql -U app -d appdb -c 'select 1'
$ docker compose down            # stops and removes containers and networks, keeps volumes
$ docker compose down -v         # also deletes volumes: this destroys the data
```

This is sample output. `up -d --build` rebuilds changed images and starts everything in the background. `exec` runs a command in a running service. `down -v` is the dangerous one: it removes named volumes, so use it deliberately.

For different environments, keep one base file and override pieces with `-f compose.prod.yaml`, or use **profiles** so optional services (a debug tool, a mail catcher) start only when requested:

```
  mailcatcher:
    image: sj26/mailcatcher
    profiles: ["dev"]
```

`docker compose --profile dev up -d` includes it, and a plain `up -d` leaves it out.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Multi-container apps with Compose fits together

Part	What it does	What goes wrong
A complete example	Read it top to bottom. Each entry under services becomes a container.	Putting passwords directly in the Compose file.
Health checks and ordering	depends_on alone only waits for the container to start, not for the program inside to be ready.	Using depends_on without a health check and assuming the dependency is ready.
Everyday commands	This is sample output. up -d --build rebuilds changed images and starts everything in the background.	Running docker compose down -v out of habit.

Flow: A complete example to Health checks and ordering / Health checks and ordering to Everyday commands

Incident: the app crashes on every first start

Incident: *After* `docker compose up -d`*, the app container exits with a connection error and restarts a few times before it works. On a slow laptop it never recovers.*

```
$ docker compose logs app | head -3
app-1  | 2026/10/24 03:12:01 connect to database: dial tcp 172.20.0.2:5432: connect: connection re \
    fused
app-1  | 2026/10/24 03:12:01 exiting
```

MARGA by EknathaLabs marga.eknathalabs.com

The app started before PostgreSQL finished initializing, and it gives up on the first failed connection. There are two layers of fix. First, the Compose file's `healthcheck` plus `condition: service_healthy` removes the race at start-up. Second, the application should **retry** its connection with a short backoff, because databases restart and networks hiccup after start-up too. Relying on start order alone would leave it fragile whenever the database restarts later.

Confirm the ordering works:

```
$ docker compose down && docker compose up -d
$ docker compose ps --format '{{.Service}} {{.Status}}'
db Up 14 seconds (healthy)
app Up 8 seconds
```

Common mistakes

- Using `depends_on` without a health check and assuming the dependency is ready
- Putting passwords directly in the Compose file
- Running `docker compose down -v` out of habit
- Expecting Compose to be a production orchestrator; it runs on one host and has no scheduling across machines

Check yourself

What does depends_on with condition: service_healthy wait for?

Answer: The dependency's health check to pass

Which command stops the stack but keeps named volumes?

Answer: docker compose down

Why mount a password as a secret file rather than an environment variable?

Answer: It does not appear in docker inspect output or the image, and it is easy to restrict

How do services reach each other in Compose?

Answer: By service name on the project network

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. Registries, tags and image supply-chain security

15 minutes. Push and pull images from registries, tag and pin them correctly, scan for vulnerabilities, generate an SBOM and sign images before they reach production.

An image is software you did not write, built from a base you did not audit, running with the privileges you give it. Registries store and distribute images, and a handful of habits keep what you pull and push trustworthy.

What you will learn

- How registries, repositories, tags and digests relate
- Tagging and pushing images, and why `latest` is a trap
- Scanning images and producing a software bill of materials (SBOM)
- Signing images and enforcing trust

Names, tags and digests

A full image reference looks like `registry.example.com/team/shop-api:1.8.2`. The registry is the server, `team/shop-api` is the repository, and `1.8.2` is a **tag**, a movable label. A tag can be repointed to different content at any time. A **digest** (`@sha256:...`) is a hash of the image content and never changes.

```
$ docker build -t registry.example.com/team/shop-api:1.8.2 .
$ docker login registry.example.com
$ docker push registry.example.com/team/shop-api:1.8.2
$ docker pull registry.example.com/team/shop-api:1.8.2
$ docker inspect --format '{{index .RepoDigests 0}}' registry.example.com/team/shop-api:1.8.2
$ docker images --digests | head -3
```

Deploy by digest when you need to be certain: `image: registry.example.com/team/shop-api@sha256:3b1c...`. Tag with the **version** and the **build identifier**, such as `1.8.2` and `1.8.2-b457`, and never deploy `latest`. It means "whatever was pushed last", so two nodes can run different code under the same name.

Choosing and pinning base images

Prefer small, maintained bases: distroless, Alpine or a slim variant of an official image. Pin the version, and for stronger reproducibility the digest.

```
FROM python:3.12-slim@sha256:0123456789abcdef0123456789abcdef0123456789abcdef0123456789abcdef
RUN useradd --create-home app
WORKDIR /app
COPY requirements.txt .
RUN python -m pip install --no-cache-dir -r requirements.txt
COPY . .
USER app
CMD ["python", "main.py"]
```

Pinning means a rebuild gives the same base. It also means you must update the pin regularly, so use an automated dependency updater to propose new digests.

Scanning and SBOMs

A scanner compares the packages inside an image against vulnerability databases. An **SBOM** (software bill of materials) lists everything in the image so you can answer "are we affected by this new CVE" in minutes.

```
$ trivy image --severity HIGH,CRITICAL registry.example.com/team/shop-api:1.8.2
$ trivy image --exit-code 1 --severity CRITICAL registry.example.com/team/shop-api:1.8.2
$ syft registry.example.com/team/shop-api:1.8.2 -o spdx-json > sbom.spdx.json
$ grype sbom:sbom.spdx.json --fail-on high
```

MARGA by EknathaLabs marga.eknathalabs.com

Run a scan in the pipeline and fail the build on critical findings with a fix available. Scan again on a schedule, because new vulnerabilities are published for images that have not changed.

Signing and verifying

Signing proves an image was built by your pipeline and has not been altered. Tools such as **cosign** (Sigstore) sign by digest and let the cluster verify before running.

```
$ cosign sign --key cosign.key registry.example.com/team/shop-api@sha256:3b1c
$ cosign verify --key cosign.pub registry.example.com/team/shop-api@sha256:3b1c
```

On the cluster side, an admission policy (for example Kyverno or a policy controller) rejects images that are unsigned or come from an unapproved registry. Without enforcement, signing is only documentation.

Registry hygiene

- Use a private registry or a pull-through cache for production
- Authenticate with short-lived credentials or workload identity, not shared passwords
- Apply retention rules to delete old untagged images
- Make production tags immutable so a tag cannot be overwritten
- Mirror critical base images, so a public outage does not stop your builds

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Registries, tags and image supply-chain security fits together

Part	What it does	What goes wrong
Names, tags and digests	A full image reference looks like registry.example.com/team/shop-api:1.8.2.	Never rebuilding images, so base-image vulnerabilities pile up.
Choosing and pinning base images	Prefer small, maintained bases: distroless, Alpine or a slim variant of an official image.	Signing images but not verifying them at admission.
Scanning and SBOMs	A scanner compares the packages inside an image against vulnerability databases.	Scanning once at build time and never again.
Signing and verifying	Signing proves an image was built by your pipeline and has not been altered.	Sharing a single registry password among all pipelines.
Registry hygiene	Use a private registry or a pull-through cache for production.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Names, tags and digests to Choosing and pinning base images / Choosing and pinning base images to Scanning and SBOMs / Scanning and SBOMs to Signing and verifying / Signing and verifying to Registry hygiene

Incident: the same tag, two behaviours

> **Incident:** *A hotfix is pushed over* `shop-api:1.8` *to fix a bug. Half the nodes already had the old layer cached and keep running it, the other half pull the new image. Customers report inconsistent behaviour, and rollback cannot be confirmed because both versions carry the same tag.*

Mutable tags make deployments unpredictable. Find which digest each Pod runs and move to immutable references:

```
$ kubectl get pods -n shop -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.status.containerS \
    tatuses[0].imageID}{"\n"}{end}'
$ kubectl set image deployment/shop-api api=registry.example.com/team/shop-api:1.8.3 -n shop
$ kubectl rollout status deployment/shop-api -n shop
```

Publish every build under a unique tag, enable tag immutability in the registry, and reference digests from the deployment manifest so the cluster can only run what the pipeline produced.

Common mistakes

- Deploying `latest` or any mutable tag
- Never rebuilding images, so base-image vulnerabilities pile up
- Scanning once at build time and never again
- Signing images but not verifying them at admission
- Sharing a single registry password among all pipelines

Check yourself

What is the difference between a tag and a digest?

Answer: A tag is a movable label; a digest is an immutable hash of the content

Why avoid deploying the latest tag?

Answer: It can point to different content over time, so nodes may run different code

What is an SBOM?

Answer: A list of all packages and components inside an image

Why is image signing not enough by itself?

Answer: Something must verify signatures at deploy time, such as an admission policy

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Container runtimes, the OCI standard and Podman

14 minutes. Understand what runs a container beneath Docker, how OCI standards make images portable, and how Podman and containerd fit in, including rootless containers.

Docker made containers popular, but Docker is not the only piece of software involved. Kubernetes does not need the Docker daemon at all. Knowing the layers helps you debug nodes, choose tools and read documentation that mentions containerd, runc or CRI.

What you will learn

- The layers: CLI, engine, high-level runtime and low-level runtime
- What the OCI image and runtime specifications guarantee
- Using Podman as a daemonless, rootless alternative
- Inspecting containers on a Kubernetes node with `crictl`

The layers

When you run `docker run`, several programs cooperate:

Layer	Example	Job
CLI	`docker`, `podman`, `nerdctl`	Takes your command
Engine / API	`dockerd`	Builds images, manages networks and volumes
High-level runtime	`containerd`, CRI-O	Pulls images, manages container lifecycle
Low-level runtime	`runc`, `crun`	Creates the Linux namespaces and cgroups and starts the process

At the bottom a container is just a Linux process started with **namespaces** (an isolated view of processes, network, mounts and users) and **cgroups** (limits on CPU and memory). `runc` is the reference tool that does exactly that.

The Open Container Initiative

The OCI publishes two specifications. The **image spec** defines how an image is laid out: a manifest, a configuration and a stack of layer tarballs. The **runtime spec** defines how a runtime turns an unpacked image into a running process. Because of these standards, an image built with Docker runs on containerd, CRI-O or Podman, and Kubernetes can swap its runtime without changing your images.

```
$ docker save shop-api:1.8.2 -o shop-api.tar
$ tar -tf shop-api.tar | head
$ docker history shop-api:1.8.2
$ docker inspect shop-api:1.8.2 --format '{{.Config.Entrypoint}} {{.Config.Cmd}}'
```

Podman

Podman runs containers without a long-running daemon, and can do so as a normal user (**rootless**). Its commands are deliberately compatible with Docker's.

```
$ podman run --rm -it docker.io/library/alpine:3.20 uname -a
$ podman build -t shop-api:dev .
$ podman ps
$ podman generate systemd --new --name shop-api > shop-api.service
$ podman pod create --name shop -p 8080:80
$ podman run -d --pod shop --name web docker.io/library/nginx:1.27
```

In rootless mode, "root" inside the container maps to your unprivileged user on the host. If an attacker escapes the container they land as an unprivileged user, which greatly limits damage. Podman can also run several containers in a **pod** that shares a network namespace, the same idea as a Kubernetes Pod,

MARGA by EknathaLabs marga.eknathalabs.com

and can export or read Kubernetes YAML with `podman generate kube` and `podman play kube`.

Runtimes under Kubernetes

Modern Kubernetes talks to a runtime through the **Container Runtime Interface (CRI)**. Most clusters use containerd or CRI-O. On a node there may be no `docker` command, so use `crictl` to see what the kubelet sees.

```
$ sudo crictl ps
$ sudo crictl images
$ sudo crictl logs 3f9a1c2b8d
$ sudo crictl inspect 3f9a1c2b8d | head -30
$ sudo crictl pods
$ systemctl status containerd
```

Isolation can also be strengthened with alternative low-level runtimes such as gVisor or Kata Containers, which add a user-space kernel or a lightweight virtual machine between the workload and the host. They trade some performance for stronger separation and are chosen with a `RuntimeClass`.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Container runtimes, the OCI standard and Podman fits together

Part	What it does	What goes wrong
The layers	When you run docker run, several programs cooperate.	Confusing the runtime spec with the image spec.
The Open Container Initiative	The OCI publishes two specifications. The image spec defines how an image is laid out: a manifest, a configuration and a stack of layer tarballs.	Mounting the Docker socket into containers, which is equivalent to root on the host.
Podman	Podman runs containers without a long-running daemon, and can do so as a normal user (rootless).	Running containers as root on the host when rootless is possible.
Runtimes under Kubernetes	Modern Kubernetes talks to a runtime through the Container Runtime Interface (CRI).	Assuming every node has the Docker daemon.

Flow: The layers to The Open Container Initiative / The Open Container Initiative to Podman / Podman to Runtimes under Kubernetes

Incident: docker command not found on the node

> **Incident:** *A Pod is stuck in* `ImagePullBackOff`*. The on-call engineer connects to the node, runs* `docker pull` *to test the registry, and gets* `command not found`*. They conclude the node is broken and drain it.*

The node is fine. It uses containerd, so there is no Docker. The kubelet's view of the problem is available with the CRI tool:

```
$ sudo crictl pull registry.example.com/team/shop-api:1.8.2
FATA[0000] pulling image: ... 401 Unauthorized
$ kubectl get secret regcred -n shop -o jsonpath='{.type}'
kubernetes.io/dockerconfigjson
```

The real cause is an expired registry credential. Refresh the image pull secret, and teach the team that `crictl` replaces `docker` on nodes, and that image pull errors are about names, authentication and network, not the node.

Common mistakes

- Assuming every node has the Docker daemon
- Running containers as root on the host when rootless is possible
- Confusing the runtime spec with the image spec

MARGA by EknathaLabs marga.eknathalabs.com

- Mounting the Docker socket into containers, which is equivalent to root on the host
- Choosing a sandbox runtime without checking it supports the features you need

Check yourself

What actually creates a container's isolation on Linux?

Answer: Namespaces and cgroups, set up by a low-level runtime such as runc

Why can an image built with Docker run on containerd or Podman?

Answer: Both follow the OCI image and runtime specifications

What is a main benefit of rootless containers?

Answer: Escaping the container lands the attacker as an unprivileged user

Which tool inspects containers on a containerd-based Kubernetes node?

Answer: crictl ps

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. Container security and troubleshooting

16 minutes. Harden containers with least privilege, scan images, and use exit codes and inspect output to find out why a container died.

A container keeps restarting. Another one passed review and then showed up in a vulnerability report. This lesson covers both sides of operating containers: reducing what an attacker can do, and finding out quickly what went wrong when something stops.

What you will learn

- A short hardening checklist for running containers
- What belongs in an image and what does not
- How to read container exit codes
- How to find out whether a container was killed for memory

Hardening checklist

Practice	How
Run as a non-root user	`USER nonroot` in the Dockerfile, or `--user 10001`
Read-only root filesystem	`--read-only`, with `--tmpfs /tmp` for scratch space
Drop Linux capabilities	`--cap-drop ALL`, then add only what you need
Block privilege escalation	`--security-opt no-new-privileges`
Limit resources	`--memory 512m --cpus 1`
Keep secrets out of images	Use runtime secrets, never `COPY` or `ENV` secrets
Pin and scan images	Use fixed tags or digests and scan for known vulnerabilities

```
docker run -d --name api \
  --user 10001:10001 \
  --read-only --tmpfs /tmp \
  --cap-drop ALL \
  --security-opt no-new-privileges \
  --memory 512m --cpus 1 \
  registry.example.com/shop-api:1.8.2
```

Warning: *Avoid* `--privileged` *and mounting* `/var/run/docker.sock` *into containers. Either gives the container nearly full control of the host, because anyone who can talk to the Docker socket can start a privileged container.*

Every package in an image is something to patch. Choose a small base, rebuild regularly for security updates, and scan in your pipeline:

```
$ trivy image --severity HIGH,CRITICAL --exit-code 1 registry.example.com/shop-api:1.8.2
```

`--exit-code 1` makes the command fail when it finds serious issues, which turns the scan into a pipeline gate. Remember that a scan shows *known* issues in the packages it can identify. It does not replace least privilege.

Why did my container stop?

Start with the state Docker recorded:

```
$ docker ps -a --filter name=api
CONTAINER ID   IMAGE          STATUS                       NAMES
8d9e0a1b2c3d   shop-api:1.8   Exited (137) 3 minutes ago   api
$ docker inspect api --format 'exit={{.State.ExitCode}} oom={{.State.OOMKilled}} restarts={{.Resta \
    rtCount}}'
exit=137 oom=true restarts=4
```

MARGA by EknathaLabs marga.eknathalabs.com

```
$ docker logs --tail 20 api
```

The exit code tells you a lot. Codes above 128 mean the process died from a signal, calculated as `128 + signal number`.

Exit code	Meaning
0	Finished normally
1	The application reported an error; read the logs
125	Docker itself failed to run the command
126	Command found but not executable
127	Command not found, often a wrong path or missing binary in the image
137	Killed by SIGKILL (9), commonly out of memory or `docker kill`
139	Segmentation fault (11)
143	Stopped by SIGTERM (15), a normal stop

For a container that will not start or exits instantly, run it interactively with a different entrypoint to explore:

```
docker run --rm -it --entrypoint sh registry.example.com/shop-api:1.8.2
```

If the image has no shell (distroless), use `docker logs`, `docker inspect`, or Docker's debug tooling to attach a shell from another image sharing the same namespaces. Do not add a shell to the production image just for convenience.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Container security and troubleshooting fits together

Part	What it does	What goes wrong
Hardening checklist	Every package in an image is something to patch.	Running with --privileged or mounting the Docker socket.
Why did my container stop?	Start with the state Docker recorded.	Restarting a crashing container repeatedly without reading the exit code.
Incident: the container is killed	The output above already contains the answer: exit=137 oom=true.	Setting no memory limit and letting one container starve the host.

Flow: Hardening checklist to Why did my container stop? / Why did my container stop? to Incident: the container is killed

Incident: the container is killed every few hours

> **Incident:** `api` *restarts about every four hours with* `Exited (137)`*. The logs show nothing unusual before the restart.*

The output above already contains the answer: `exit=137 oom=true`. The kernel killed the process for exceeding the container's memory limit, so the application had no chance to log anything. To see the trend instead of guessing:

```
$ docker stats --no-stream api
CONTAINER ID   NAME   CPU %   MEM USAGE / LIMIT   MEM %    NET I/O
8d9e0a1b2c3d   api    12.4%   498MiB / 512MiB     97.3%    1.2GB / 800MB
```

Memory sits at 97% of the 512 MiB limit. Two honest options exist. If the service legitimately needs more memory under load, raise the limit after checking peak usage. If memory grows steadily until it dies, you have a leak: capture a heap profile or trend memory in your metrics, fix the code, and keep the limit as a safety net. Raising the limit repeatedly without watching the trend only delays the next kill. Also set a

MARGA by EknathaLabs marga.eknathalabs.com

restart policy such as `--restart unless-stopped` so a crash does not become an outage, while still alerting on restart counts so you notice.

Common mistakes

- Running with `--privileged` or mounting the Docker socket
- Adding secrets with `ENV` or `COPY`
- Restarting a crashing container repeatedly without reading the exit code
- Setting no memory limit and letting one container starve the host

Check yourself

A container shows Exited (137) and OOMKilled=true. What happened?

Answer: The kernel killed it with SIGKILL because it exceeded its memory limit

Which flag set reduces the privileges of a container the most among these?

Answer: --cap-drop ALL --security-opt no-new-privileges --read-only

Why is mounting /var/run/docker.sock into a container dangerous?

Answer: Access to the Docker API allows starting privileged containers, which gives control of the host

What does exit code 127 usually mean?

Answer: The command was not found, such as a wrong path or a missing binary in the image

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 8. BuildKit, build caching and multi-architecture images

13 minutes. Make image builds fast and repeatable with layer ordering, BuildKit cache mounts, secrets that do not leak into layers, remote caches and images that run on both amd64 and arm64.

A slow image build slows every developer and every pipeline run. A careless one leaks secrets into layers or produces an image that cannot run on the machine where it is deployed. BuildKit, the modern build engine, gives you precise control over caching, secrets and target platforms.

What you will learn

- How layer caching works and how to order instructions
- Cache mounts for package managers
- Build secrets and SSH forwarding without leaking into layers
- Sharing cache between pipeline runs
- Building for amd64 and arm64 with `buildx`

How the cache works

Each instruction in a Dockerfile creates a layer. If the instruction and everything before it are unchanged, the layer comes from cache. As soon as one layer changes, all following layers are rebuilt. So order instructions from least to most frequently changing:

```
# syntax=docker/dockerfile:1
FROM python:3.12-slim AS build
WORKDIR /app
COPY requirements.txt .                     # changes rarely
RUN --mount=type=cache,target=/root/.cache/pip \
    pip install --no-cache-dir -r requirements.txt
COPY . .                                    # changes on every commit
RUN python -m compileall .
```

Copying the dependency list first means a code change does not reinstall the dependencies. A `.dockerignore` file keeps unrelated changes (test output, local files) from invalidating the `COPY . .` layer.

Cache mounts

`RUN --mount=type=cache` keeps a directory (pip, npm, apt, Go build cache) between builds without storing it in the image. Builds become much faster, and the image stays small.

Secrets

Never use `ARG` or `ENV` for a secret, or `COPY` a credentials file: they stay in image layers and history. Use a build secret, which is mounted only during one `RUN`:

```
FROM node:20-slim
RUN --mount=type=secret,id=npmrc,target=/root/.npmrc npm ci
```

```
docker build --secret id=npmrc,src=$HOME/.npmrc -t shop:1.0 .
docker history --no-trunc shop:1.0 | grep -i npmrc || echo "not in history"
```

For private repositories use `--mount=type=ssh` and `--ssh default` so the key never enters the image.

Cache in pipelines

Pipeline runners start clean, so local layer cache is empty. Export and import the cache:

```
docker buildx build \
  --cache-from type=registry,ref=registry.example.com/shop:buildcache \
  --cache-to   type=registry,ref=registry.example.com/shop:buildcache,mode=max \
  -t registry.example.com/shop:1.4.2 --push .
```

MARGA by EknathaLabs marga.eknathalabs.com

Multi-architecture images

Teams now run on both amd64 servers and arm64 machines (developer laptops, cloud processors that cost less). An image built for one fails on the other with "exec format error". Build a manifest list covering both:

```
docker buildx create --use
docker buildx build --platform linux/amd64,linux/arm64 -t registry.example.com/shop:1.4.2 --push .
docker buildx imagetools inspect registry.example.com/shop:1.4.2   # shows each platform
```

Compile inside the build for the target (for example `GOARCH=$TARGETARCH`), and test the arm64 image, not only that it builds. Emulation with QEMU works but is slow, so use native builders when you can.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How BuildKit, build caching and multi-architecture... fits together

Part	What it does	What goes wrong
How the cache works	Each instruction in a Dockerfile creates a layer.	COPY . . before dependency installation.
Cache mounts	RUN --mount=type=cache keeps a directory (pip, npm, apt, Go build cache) between builds without storing it in the image.	Publishing a single-architecture image to mixed fleets.
Secrets	Never use ARG or ENV for a secret, or COPY a credentials file: they stay in image layers and history.	Secrets in ARG, ENV or files copied into layers.
Cache in pipelines	Pipeline runners start clean, so local layer cache is empty.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Multi-architecture images	Teams now run on both amd64 servers and arm64 machines (developer laptops, cloud processors that cost less).	An image built for one fails on the other with "exec format error".

Flow: How the cache works to Cache mounts / Cache mounts to Secrets / Secrets to Cache in pipelines / Cache in pipelines to Multi-architecture images

Incident: the token in the layer

> **Incident:** *A security scan finds a package registry token in a public image, though the Dockerfile's last line removes the file.*

```
$ docker history --no-trunc shop:0.9 | grep -i NPM_TOKEN
 ARG NPM_TOKEN=npm_xxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

The token was passed as a build argument and a `RUN rm` in a later layer does not remove earlier layers. The team revoked the token, rebuilt with a build secret, scanned the history in CI, and added a rule against `ARG` names that contain TOKEN or KEY.

Common mistakes

- `COPY . .` before dependency installation
- Secrets in `ARG`, `ENV` or files copied into layers
- No `.dockerignore`
- Publishing a single-architecture image to mixed fleets

Check yourself

What happens to the cache when one layer changes?

Answer: That layer and all later layers are rebuilt

MARGA by EknathaLabs marga.eknathalabs.com

How should a build use a private registry token?

Answer: A build secret mounted for one RUN instruction

What does RUN --mount=type=cache provide?

Answer: A directory reused between builds that is not part of the image

What error shows an image was built for the wrong CPU architecture?

Answer: exec format error

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 9. Container logging, health checks and resource limits

13 minutes. Log to standard output, control log growth, add meaningful health checks, and set CPU, memory and process limits that protect the host without causing surprise restarts.

Containers run for months and fail at awkward times. The three things that decide whether you can cope are: whether you can see what happened (logs), whether the platform can tell if the app is healthy (health checks), and whether one container can harm its neighbours (limits).

What you will learn

- Why containers log to standard output and how logging drivers work
- Preventing logs from filling the disk
- Health checks that tell the truth
- CPU, memory, process and file limits
- Restart policies and their traps

Logging

A container's main process should write logs to standard output and standard error, not to files inside the container. The runtime captures them, and `docker logs` or the platform collects them. Files inside the container are lost with the container and fill its writable layer.

By default the `json-file` driver stores logs without a size limit, so a chatty service can fill the host's disk. Set rotation:

```
{ "log-driver": "json-file", "log-opts": { "max-size": "10m", "max-file": "3" } }
```

in `/etc/docker/daemon.json`, or per container with `--log-opt max-size=10m --log-opt max-file=3`. For central logging, use a driver or an agent that ships logs to your log system. Write structured (JSON) logs with a request id so they can be searched.

Health checks

```
FROM alpine:3.20
HEALTHCHECK --interval=15s --timeout=3s --start-period=30s --retries=3 \
  CMD wget -qO- http://127.0.0.1:8080/health || exit 1
```

A good health endpoint checks that the app can do its job (for example, reach its database), but not so much that a slow dependency makes every instance unhealthy at once. Separate readiness (can it take traffic) from liveness (should it be restarted). A `start-period` stops slow-starting apps from being killed before they are ready.

Limits

```
docker run -d --name shop \
  --memory 512m --memory-swap 512m \
  --cpus 1.0 \
  --pids-limit 200 \
  --ulimit nofile=4096:4096 \
  --read-only --tmpfs /tmp \
  shop:1.4.2
```

- `--memory` is a hard limit: going over it ends in an OOM kill (exit code 137). Set `--memory-swap` equal to it to disable swap use.
- `--cpus` throttles; it does not kill. Throttling appears as latency.
- `--pids-limit` stops fork bombs and thread leaks.
- `--ulimit nofile` limits open files.
- `--read-only` with `--tmpfs` limits what an attacker or a bug can write.

MARGA by EknathaLabs marga.eknathalabs.com

Choose limits from measured use: run under realistic load, look at `docker stats`, and add headroom. Limits that are too tight cause restarts; none at all lets one container starve the host.

Restart policies

`--restart unless-stopped` restarts a container that exits. This helps with rare crashes but hides a crash loop. Always alert on restart counts, and pair restarts with logs that show why the process exited.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Container logging, health checks and resource... fits together

Part	What it does	What goes wrong
Logging	A container's main process should write logs to standard output and standard error, not to files inside the container.	Logging to files inside containers.
Health checks	A good health endpoint checks that the app can do its job (for example, reach its database), but not so much that a slow dependency makes every instance unhealthy at...	Health checks that check nothing, or everything.
Limits	--memory is a hard limit: going over it ends in an OOM kill (exit code 137).	Check the Common mistakes list below this diagram for what usually goes wrong here.
Restart policies	--restart unless-stopped restarts a container that exits.	Restart policies without restart alerts.

Flow: Logging to Health checks / Health checks to Limits / Limits to Restart policies

Incident: the disk that filled with logs

Incident: *A host's disk reaches 100 percent overnight and all containers on it stop working.*

```
$ du -sh /var/lib/docker/containers/*/*-json.log | sort -h | tail -2
48G  /var/lib/docker/containers/9f1c.../9f1c...-json.log
$ docker logs --tail 3 api
DEBUG cache miss key=...
```

A debug log level had been left on, and the default log driver had no size limit. The team set rotation in the daemon configuration, moved to structured logs sent to the central system, and added a disk usage alert at 80 percent.

Common mistakes

- Logging to files inside containers
- No log rotation
- Health checks that check nothing, or everything
- Restart policies without restart alerts

Check yourself

Where should a containerised application write its logs?

Answer: Standard output and standard error

What happens when a container exceeds its memory limit?

Answer: The kernel kills it, usually exit code 137

Which Docker option stops a runaway process from forking without end?

Answer: --pids-limit

Why is a start-period useful in a health check?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: It gives slow-starting apps time before failures count

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 10. Slim images: smaller, faster and safer

12 minutes. Shrink container images with the right base image, multi-stage builds and clean layers, and see what is inside an image.

An image of 1.4 GB takes minutes to pull on every new node, contains compilers an attacker could use, and shows hundreds of vulnerabilities in a scan. Most of that weight is not your application. Small images start faster, cost less to store and give an attacker less to work with.

What you will learn

- Choose a base image on purpose: full, slim, alpine or distroless
- Use multi-stage builds to leave build tools behind
- Keep layers small and avoid hidden bulk
- Inspect an image to see where the size comes from

Pick the base on purpose

The base image is usually most of the size. Typical choices:

Base	Size	Notes
`python:3.12`	about 1 GB	Compilers and tools, good for building
`python:3.12-slim`	about 150 MB	Debian without most extras, a safe default
`python:3.12-alpine`	about 50 MB	Uses musl, which can break some compiled packages
distroless	20 to 60 MB	No shell or package manager, hardest to attack

Alpine is small but uses a different C library, and some Python wheels then have to be built from source. Slim images are the practical middle. Distroless images are excellent for production, but without a shell you debug with an ephemeral debug container or a separate debug tag.

Multi-stage builds

Build tools are needed to create the program, not to run it. A multi-stage Dockerfile builds in one stage and copies only the result into a small final stage.

```
FROM golang:1.22 AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -o /out/app ./cmd/app

FROM gcr.io/distroless/static-debian12
COPY --from=build /out/app /app
USER nonroot
ENTRYPOINT ["/app"]
```

The final image holds one static binary. The compiler, the source code and the module cache stay in the first stage and are not part of what you ship.

Layers and what hides in them

Every instruction can add a layer, and a file deleted in a later layer still takes space in the earlier one. Clean up in the same instruction that creates the mess.

```
RUN apt-get update \
 && apt-get install -y --no-install-recommends curl ca-certificates \
 && rm -rf /var/lib/apt/lists/*
```

MARGA by EknathaLabs marga.eknathalabs.com

Use a `.dockerignore` file so the build context does not include `node_modules`, test data, local virtual environments or secrets. Copy the files that change least first (dependency lists), so the cache is reused when only source code changes.

See where the size comes from

Measure before you optimize.

```
$ docker images shop
REPOSITORY   TAG    SIZE
shop         1.0    1.38GB
$ docker history shop:1.0 | head
$ docker run --rm -it wagoodman/dive shop:1.0
```

`docker history` lists layers with their sizes, and tools such as `dive` show which files each layer added. A scan with Trivy or Grype on the slim image typically reports far fewer packages, which means fewer alerts to triage.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Slim images: smaller, faster and safer fits together

Part	What it does	What goes wrong
Pick the base on purpose	The base image is usually most of the size.	Choosing Alpine for Python and then fighting native build errors.
Multi-stage builds	Build tools are needed to create the program, not to run it.	Using the full language image as the final image.
Layers and what hides in them	Every instruction can add a layer, and a file deleted in a later layer still takes space in the earlier one.	Deleting files in a later layer and expecting the size to drop.
See where the size comes from	Measure before you optimize.	Copying the whole project, including secrets and caches, into the image.

Flow: Pick the base on purpose to Multi-stage builds / Multi-stage builds to Layers and what hides in them / Layers and what hides in them to See where the size comes from

Incident: A 1.4 GB image on every node

Incident: *Scale-out events took ten minutes because each new node pulled the application image from the registry first.*

```
$ docker history shop:1.0 --format "{{.Size}}\t{{.CreatedBy}}" | head -4
610MB   RUN /bin/sh -c apt-get install -y build-essential
480MB   COPY . /app
210MB   RUN pip install -r requirements.txt
```

The image carried compilers and the whole repository, including test data and a local virtual environment. A multi-stage build, a slim base, and a `.dockerignore` brought it down to 170 MB. Pulls dropped from minutes to seconds, and the vulnerability scan went from over 400 findings to 31.

Common mistakes

- Using the full language image as the final image
- Deleting files in a later layer and expecting the size to drop
- Copying the whole project, including secrets and caches, into the image
- Choosing Alpine for Python and then fighting native build errors

Check yourself

What does a multi-stage build achieve?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: The final image contains only what it needs at run time

Why does deleting a file in a later layer not shrink the image?

Answer: The earlier layer still stores it

What does a .dockerignore file do?

Answer: Keeps files out of the build context

Which command lists the layers and sizes of an image?

Answer: docker history

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 11. Running containers in production

13 minutes. Run containers with the settings production needs: non-root users, read-only filesystems, health checks, limits, restart policy and graceful shutdown.

A container that works on a laptop can still be a poor citizen in production. It runs as root, it ignores the stop signal and is killed after ten seconds mid-request, and it has no limits, so one bug takes the host down. A short checklist of run-time settings turns a working container into a dependable one.

What you will learn

- Run as a non-root user with a read-only filesystem and dropped capabilities
- Handle SIGTERM for graceful shutdown
- Set resource limits, health checks and a restart policy
- Log to standard output and keep data in volumes

Reduce what a container may do

Containers share the host kernel, so you narrow their rights. Four settings do most of the work:

```
$ docker run -d --name shop \
    --user 10001:10001 \
    --read-only --tmpfs /tmp \
    --cap-drop ALL \
    --security-opt no-new-privileges \
    shop:1.4
```

`--user` avoids root inside the container, `--read-only` stops an attacker from changing the program, `--tmpfs /tmp` provides the one writable place most programs need, and `--cap-drop ALL` removes Linux capabilities that a web service never uses. Add back only what you must, for example `--cap-add NET_BIND_SERVICE` to bind a port below 1024.

Stop gracefully

`docker stop` sends SIGTERM and waits ten seconds before SIGKILL. A service must catch SIGTERM, stop accepting new work, finish current requests and exit. In a Dockerfile, use the exec form so the program is process 1 and receives the signal:

```
ENTRYPOINT ["/app/server"]        # good: the server gets SIGTERM
# ENTRYPOINT /app/server          # shell form: a shell is PID 1 and may swallow it
```

If the program needs longer than ten seconds to drain, raise the timeout with `docker stop -t 30` or `stop_grace_period` in Compose. If it is a script, use `exec` at the end so the shell is replaced by the program, or use a tiny init such as `tini` (`docker run --init`).

Limits, health and restarts

```
$ docker run -d --name shop \
    --memory 512m --cpus 1.0 --pids-limit 200 \
    --restart unless-stopped \
    --health-cmd "curl -fsS http://localhost:8080/healthz || exit 1" \
    --health-interval 15s --health-retries 3 \
    shop:1.4
```

The memory limit makes the kernel kill only this container if it grows without bound. The pids limit stops a fork bomb. A restart policy brings the container back after a crash or reboot, and a health check lets the platform see that a running process is no longer serving. Health checks should test the application, not just the port.

MARGA by EknathaLabs marga.eknathalabs.com

Logs and data

Write logs to standard output and standard error and let the runtime collect them; do not write log files inside the container. Set a log driver limit so the host disk cannot fill (`--log-opt max-size=10m --log-opt max-file=3`). Keep data that must survive in a **volume**, and treat the container filesystem as disposable. Pin image tags to a version or a digest, never `latest`, so a restart does not silently change the program.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Running containers in production fits together

Part	What it does	What goes wrong
Reduce what a container may do	Containers share the host kernel, so you narrow their rights.	Running as root with a writable filesystem.
Stop gracefully	docker stop sends SIGTERM and waits ten seconds before SIGKILL.	Using the shell form of ENTRYPOINT or CMD.
Limits, health and restarts	The memory limit makes the kernel kill only this container if it grows without bound.	Skipping resource limits and health checks.
Logs and data	Write logs to standard output and standard error and let the runtime collect them; do not write log files inside the container.	Writing logs and data inside the container filesystem.

Flow: Reduce what a container may do to Stop gracefully / Stop gracefully to Limits, health and restarts / Limits, health and restarts to Logs and data

Incident: Requests lost on every deploy

Incident: *Each rolling restart produced a burst of 502 errors for about ten seconds.*

```
$ docker stop shop; echo $?
137
$ docker logs shop | tail -2
listening on :8080
(no shutdown message)
```

Exit status 137 means the container was killed. The Dockerfile used the shell form of `ENTRYPOINT`, so the application never received SIGTERM, and the in-flight requests were cut off. After switching to the exec form and adding a handler that stops accepting connections and drains for up to 20 seconds, the 502 bursts disappeared. The team also set a `stop_grace_period` of 30 seconds.

Common mistakes

- Running as root with a writable filesystem
- Using the shell form of ENTRYPOINT or CMD
- Skipping resource limits and health checks
- Writing logs and data inside the container filesystem

Check yourself

What does exit status 137 usually mean for a container?

Answer: The process was killed by SIGKILL, often after ignoring SIGTERM or running out of memory

Why use the exec form of ENTRYPOINT?

Answer: The program becomes PID 1 and receives signals directly

Which options reduce what a compromised container can do?

Answer: Non-root user, read-only filesystem, dropped capabilities

Why set a health check?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: So the platform can detect that a running process is no longer serving

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 12. Debugging containers that will not start or keep dying

15 minutes. Read exit codes, logs and events, inspect a container's config, and get a shell inside a failing or minimal image without guessing.

A container that exits straight away, restarts in a loop, or behaves differently from your laptop frustrates people because it leaves little behind. In fact Docker keeps a lot of evidence: the exit code, the last logs, the configuration it actually used, and events. This lesson shows where each piece lives.

What you will learn

- Reading exit codes (0, 1, 125-127, 137, 139, 143)
- Using `logs`, `inspect`, `events`, `stats` and `exec`
- Debugging images with no shell
- The usual causes of crash loops

Start with the facts

```
$ docker ps -a --format 'table {{.Names}}\t{{.Status}}'
NAMES    STATUS
shop     Restarting (137) 12 seconds ago
$ docker logs --tail 50 --timestamps shop
$ docker inspect shop --format '{{.State.ExitCode}} {{.State.OOMKilled}} {{.HostConfig.Memory}}'
137 true 268435456
```

Exit code	Usual meaning
0	Process finished normally
1	Application error
125	Docker itself failed to run the command
126, 127	Command not executable, or not found
137	Killed (SIGKILL), often out of memory
139	Segmentation fault
143	Stopped by SIGTERM

Here 137 together with `OOMKilled true` and a 256 MB limit says the process used more memory than allowed.

Get inside

```
docker exec -it shop sh                      # shell in a running container
docker run --rm -it --entrypoint sh shop:1.4  # shell in the image, skipping the failing entrypoin \
    t
docker cp shop:/app/config.yml .             # copy a file out to read it
docker diff shop                             # what changed in the filesystem
docker stats --no-stream                     # CPU and memory now
docker events --since 10m                    # starts, dies, OOM and health events
```

For minimal images with no shell, attach a debug container that shares the target's namespaces:

```
docker run --rm -it --pid=container:shop --net=container:shop nicolaka/netshoot
```

Usual causes of a crash loop

- The main process exits (a script that ends, or a service that daemonises itself)
- A missing environment variable or config file
- A port already in use, or binding to `127.0.0.1` so published ports do not work
- The wrong user, so the app cannot write to a volume

MARGA by EknathaLabs marga.eknathalabs.com

- A health check that is too strict, so the orchestrator keeps killing a working container

Diagram: From a dead container to a cause

Part	What it does	What goes wrong
Exit code and state	docker ps -a and inspect give the code and whether it was OOM killed.	Looking only at the restart count hides why it exits.
Logs	docker logs shows the last output with timestamps.	Applications that log to files inside the container leave nothing here.
Effective config	inspect shows env, mounts, user, limits and command.	Assuming the image defaults still apply misses overrides.
Shell inside	exec or run --entrypoint sh lets you test files and commands.	Minimal images lack a shell; use a debug container.
Fix the image or run options	Change the cause, rebuild, and re-run with the same flags.	Fixing only in a running container is lost on the next start.

Flow: Exit code and state to Logs / Exit code and state to Effective config / Logs to Shell inside / Effective config to Shell inside / Shell inside to Fix the image or run options

Incident: the container that restarted every minute

> **Incident:** *A new image is released and the service restarts about once a minute. Logs show nothing unusual before each restart.*

```
$ docker inspect api --format '{{.State.OOMKilled}} {{.HostConfig.Memory}}'
true 536870912
$ docker stats --no-stream api
NAME   CPU %   MEM USAGE / LIMIT
api    3.1%    508MiB / 512MiB
```

A new cache library kept results without a size limit. Memory grew until the 512 MB limit killed the process with code 137. The team bounded the cache, raised the limit with measured headroom, and added a memory alert.

Common mistakes

- Running `docker logs` once, seeing nothing, and giving up (check the exit code and events)
- Debugging in a running container and forgetting to put the fix into the image
- Using `latest` tags, so nobody knows which image is running
- Treating a restart policy as a fix

Check yourself

A container exits with code 137 and OOMKilled is true. What happened?

Answer: The kernel killed it for exceeding its memory limit

How do you open a shell in an image whose entrypoint crashes immediately?

Answer: docker run --rm -it --entrypoint sh image

Which command shows the configuration a container actually runs with?

Answer: docker inspect

A service inside a container listens on 127.0.0.1 and the published port does not work. Why?

Answer: Traffic from outside arrives on the container's other interface, so the app must listen on 0.0.0.0

MARGA by EknathaLabs marga.eknathalabs.com

Cheat sheet: Docker and Compose

Run, build, inspect, clean up and harden containers, plus everyday Compose commands.

Run and manage

```
docker run -d --name web -p 8080:80 nginx:1.27
docker run --rm -it alpine:3.20 sh          # throwaway interactive container
docker ps -a                                 # include stopped
docker stop web && docker rm web
docker exec -it web sh                       # shell in a running container
docker logs -f --tail 50 web
docker cp web:/etc/nginx/nginx.conf ./nginx.conf
```

Flag	Meaning
`-d`	Run in the background
`-p HOST:CONTAINER`	Publish a port
`-e KEY=value`	Set an environment variable
`-v NAME:/path`	Mount a volume
`--network NAME`	Join a network
`--restart unless-stopped`	Restart policy
`--memory 512m --cpus 1`	Resource limits
`--user 10001`	Run as a non-root user

Build and ship

```
docker build -t shop-api:1.8.2 .
docker build --no-cache -t shop-api:dev .
docker tag shop-api:1.8.2 registry.example.com/shop-api:1.8.2
docker push registry.example.com/shop-api:1.8.2
docker pull registry.example.com/shop-api:1.8.2
docker history shop-api:1.8.2
```

Dockerfile essentials

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
USER 10001
EXPOSE 8080
CMD ["python", "app.py"]
```

Rule	Why
Copy dependency files first	Keeps the install layer cached
Pin base image versions	Reproducible builds
Use multi-stage builds	Small final image without build tools
Use exec form for `CMD`	Program is PID 1 and receives signals
Add `.dockerignore`	Smaller context, no secrets copied

Volumes and networks

```
docker volume create pgdata
docker volume ls
docker volume inspect pgdata
docker network create backend
docker network connect backend web
docker network inspect backend
```

On a user-defined network containers resolve each other by name. The default bridge does not.

MARGA by EknathaLabs marga.eknathalabs.com

Inspect and debug

```
docker inspect web --format '{{.State.ExitCode}} oom={{.State.OOMKilled}}'
docker inspect web --format '{{range .NetworkSettings.Networks}}{{.IPAddress}} {{end}}'
docker stats --no-stream
docker top web
docker events --since 10m
docker run --rm -it --entrypoint sh shop-api:1.8.2
```

Exit code	Meaning
0	Finished normally
1	Application error
125 / 126 / 127	Docker error / not executable / command not found
137	SIGKILL, often out of memory
139	Segmentation fault
143	SIGTERM, normal stop

Clean up

```
docker system df                    # disk used by images, containers, volumes
docker container prune              # remove stopped containers
docker image prune -a               # remove unused images
docker volume prune                 # remove unused volumes (check first)
```

Compose

```
docker compose up -d --build
docker compose ps
docker compose logs -f --tail 50 app
docker compose exec db psql -U app -d appdb
docker compose config               # validate and print the merged file
docker compose down                 # keeps volumes
docker compose down -v              # also deletes volumes: destroys data
```

Hardened run

```
docker run -d --name api \
  --user 10001:10001 --read-only --tmpfs /tmp \
  --cap-drop ALL --security-opt no-new-privileges \
  --memory 512m --cpus 1 \
  registry.example.com/shop-api:1.8.2
```

Avoid `--privileged` and mounting `/var/run/docker.sock` into containers.

Images and registries, more

```
docker image ls --digests
docker history --no-trunc app:1.4
docker image inspect app:1.4 --format '{{.Config.User}} {{.Config.Entrypoint}}'
docker tag app:1.4 registry.example.com/team/app:1.4
docker push registry.example.com/team/app:1.4
docker save -o app.tar app:1.4
docker load -i app.tar
docker buildx build --platform linux/amd64,linux/arm64 -t app:1.4 .
docker scout cves app:1.4
```

Warning: *Do not tag production deployments with* `latest`*. It moves, so two hosts can run different code. Pin a version tag or, better, a digest.*

Warning: *Never copy secrets into an image with* `COPY` *or* `ARG`*. They stay in the image layers and in* `docker history`*. Use build secrets (*`--secret`*) or runtime injection.*

MARGA by EknathaLabs marga.eknathalabs.com

Run and inspect containers, more

```
docker run -d --name web --restart unless-stopped -p 8080:80 nginx:1.27
docker run --rm -it --user 1000:1000 --read-only --cap-drop ALL alpine sh
docker run --memory 512m --cpus 1.5 app:1.4
docker exec -it web sh
docker logs -f --tail 100 --since 10m web
docker inspect web --format '{{.State.Health.Status}} {{.NetworkSettings.IPAddress}}'
docker stats --no-stream
docker top web
docker diff web
docker cp web:/etc/nginx/nginx.conf ./nginx.conf
docker port web
docker events --since 10m
```

Warning: `--privileged` *gives a container nearly full access to the host. Do not use it unless you understand why, and never mount* `/var/run/docker.sock` *into an untrusted container: it is root on the host.*

Warning: *Publishing with* `-p 8080:80` *listens on all interfaces and Docker edits firewall rules itself. Bind to localhost with* `-p 127.0.0.1:8080:80` *when the service is private.*

Volumes and networks

```
docker volume ls
docker volume inspect pgdata
docker run -v pgdata:/var/lib/postgresql/data postgres:16
docker run -v "$PWD":/work:ro alpine ls /work
docker network ls
docker network create --driver bridge backend
docker network connect backend web
docker network inspect backend
```

Warning: *Data written inside a container is lost when the container is removed. Keep state in named volumes and back them up.*

Warning: *A bind mount such as* `-v /:/host` *exposes the host filesystem. Mount only what you need, and read-only (*`:ro`*) where possible.*

Compose

```
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs -f api
docker compose exec api sh
docker compose pull && docker compose up -d
docker compose down
```

Danger: `docker compose down -v` *also deletes the named volumes, including your database. Use plain* `down` *unless you really want to wipe the data.*

Cleanup and cautions

```
docker system df
docker container prune
docker image prune
docker builder prune --filter until=168h
```

Danger: `docker system prune -a --volumes` *removes every unused image, stopped container and unused volume. On a shared host that can delete someone else's data. Look at* `docker system df` *and prune one resource type at a time.*

Warning: `docker rm -f` *kills a running container without a graceful stop. Use* `docker stop` *first, which sends SIGTERM and waits.*

MARGA by EknathaLabs
marga.eknathalabs.com